

Practical 3

AIM:. To Implementation of max-heap sort algorithm

1 ALGORITHM:

Algorithm:-
 Input:- array elements.
 output:- sorted array.
 steps:-
 step 1:- start
 step 2:- for $i = n/2$ to 1 to 0
 step 3:- largest $\leftarrow i$
 step 4:- $i \leftarrow 2 * i$
 step 5:- if $i < 2 * i + 1$
 step 6:- if $i < n$ and $arr[i] > arr[largest]$
 then do largest $\leftarrow i$
 step 7:- if $i < n$ and $arr[i] > arr[largest]$
 then do largest $\leftarrow i$
 step 8:- if largest $\neq i$ then do
 step 9:- swap $arr[largest]$ and $arr[i]$
 step 10:- For $i \leftarrow i - 1$ to 1
 step 11:- swap $arr[i]$ and $arr[i]$
 step 12:- Give $i = 1$ and
 step 13:- max heapies $[arr, i-1]$
 step 14:- stop.

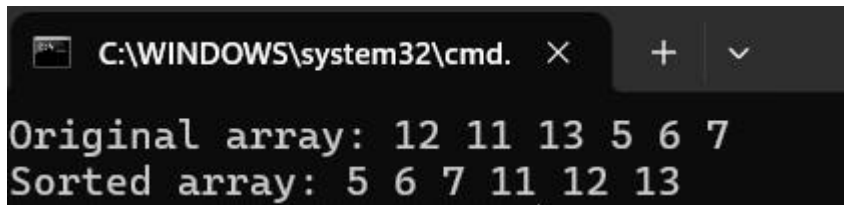
2 PROGRAM:

```
#include <iostream> using
namespace std; void heapify(int
arr[], int n, int i)
{ int largest = i; int
left = 2 * i + 1; int
right = 2 * i + 2; if
(left < n &&
```

```
arr[left] >
arr[largest]) largest
= left; if (right < n
&& arr[right] >
arr[largest]) largest
= right; if (largest
!= i){ int temp =
arr[i]; arr[i] =
arr[largest];
arr[largest] = temp;
heapify(arr, n,
largest); }} void
heapSort(int arr[],
int n){ // Build
Max Heap for (int
i= n/2-1;i >= 0;i--)
heapify(arr, n, i); //
Heap Sort for (int
i=n-1;i>0;i--){ int
temp = arr[0];
arr[0] = arr[i];
arr[i] = temp;
heapify(arr, i, 0);}}
int main(){ int arr[]
= {12, 11, 13, 5, 6,
7}; int n = 6; cout
<< "Original array:
"; for (int i = 0; i <
n; i++) cout <<
arr[i] << " ";
```

```
heapSort(arr, n);
cout << "\nSorted
array: "; for (int i =
0; i < n; i++) cout
<< arr[i] << " ";
return 0;}
```

3 OUTPUT:



4 TIME COMPLEXITY:

Time complexity! (Best, Average, worst)

for ($i = n/2$; $i > 1$; $i--$) $\rightarrow n$
 $\text{maxHeapify(arr, n, i)} \rightarrow \log n$
 for ($i = n$; $i > 1$; $i--$) $\rightarrow n$
 $\text{swap(arr[i], arr[1])} \rightarrow 1$
 $\text{maxHeapify(arr, i-1, i)} \rightarrow \log n$
 $\text{Time} = n \times \log n \times n \times 1 \times \log n$
 $= n \log n + n \log n$
 $\text{Time} = O(n \log n)$
 $\text{Best} = O(n \log n)$

worst case!

$T(\text{worst}(n)) = T(n/2) + \alpha(1)$
 $a=1, b=2, f(n)=1$
 $n \log_b a = 1$
 $T(\text{worst}(n)) = O(n \log n)$

Average case!

$T_{\text{avg}} = T(n/2) + \alpha(1)$
 $n \log_b 2 = 1$
 $T_{\log}(n) = O(n \log n)$

5 Conclusion:

The Max-Heap Sort algorithm was successfully implemented and executed. The given elements were sorted in ascending order, and the time complexity was analyzed for the best, average, and worst cases. Thus, the implementation of the Max-Heap Sort algorithm was successfully done.